

Complementary Branching for #SAT

Bounded-Degree Bounds and Faster #3-SAT

Konstantin Kutzkov | Archon Labs | kutzkov@gmail.com | IJCAI 2026

Propositional Model Counting

#SAT asks for the number of models of a CNF formula F over n variables,

$$\#SAT(F) = |\{\sigma : \sigma \models F\}|,$$

and is #P-complete [Valiant '79]. Exact algorithms *branch*: split F , solve the parts, combine the counts. Counting is over *all* n variables — a formula with no clauses has 2^n models.

Running example

$$F = \underbrace{(\bar{a} \vee x_1 \vee x_2)}_{C_1} \wedge \underbrace{(\bar{a} \vee x_3 \vee x_4)}_{C_2} \wedge \underbrace{(a \vee x_1 \vee y)}_{C_3} \\ \wedge \underbrace{(x_2 \vee \bar{x}_3)}_{C_4} \wedge \underbrace{(\bar{x}_1 \vee x_4 \vee y)}_{C_5}$$

$n = 6, \quad m = 5, \quad \#SAT(F) = 28.$

a has degree 3 and y is *pure* — it occurs only positively.

Complement Counting

For any formulas G, H , partition the models of G by whether they satisfy H :

$$\#SAT(G) = \#SAT(G \wedge H) + \#SAT(G \wedge \neg H) \implies$$

$$\#SAT(G \wedge H) = \#SAT(G) - \#SAT(G \wedge \neg H)$$

Instead of *satisfying* a constraint, count without it and **subtract the violators**. The relaxed instance $\#SAT(G)$ is cheap, and $\neg H$ is a DNF we can branch on. Only a disjoint partition of assignments is used, so **weighted** counting comes for free.

Three choices of H give three branching rules — each with its own result.

1. Clause Branching $H = C$, a single clause

With $F = G \wedge C$,

$$\#SAT(F) = \underbrace{\#SAT(G)}_{\text{remove } C} - \underbrace{\#SAT(G \wedge \neg C)}_{\text{falsify } C}.$$

- **Remove** C : one clause fewer, no variable assigned.
- **Falsify** C : $\neg C$ is a conjunction of *unit* literals — it assigns $|C|$ variables at once.

► $C = C_4 = (x_2 \vee \bar{x}_3)$, so $\neg C_4 = \bar{x}_2 \wedge x_3$:

$$\underbrace{28}_{\#SAT(F)} = \underbrace{36}_{\#SAT(G)} - \underbrace{8}_{\#SAT(G \wedge \neg C_4)}.$$

Result

#SAT can be solved in time $\mathcal{O}^*(2^m)$ and polynomial space — inclusion–exclusion [Lozinskii '92] without the alternating signs.

Takeaway

Complementary branching decomposes #SAT by *negating* subformulas instead of assigning variables, and naturally augments classical DPLL branching. On the MC2025 benchmarks, low-degree variables in long clauses and low-degree pure variables are abundant, and the largest instances benefit the most.

A #SAT solver based on complementary counting is currently being developed, please check <https://github.com/konstantinkutzkov/COCO> for the latest version.

2. DNF Branching $H = F_a$, all clauses containing a

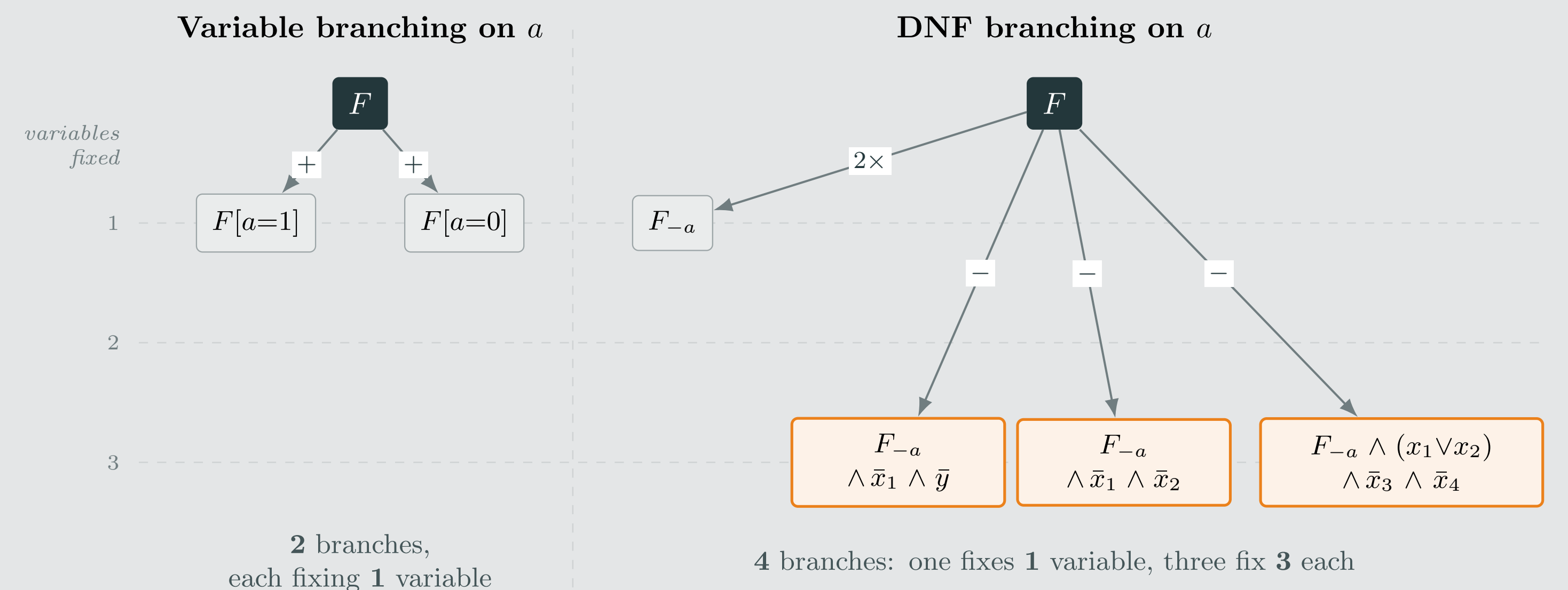
Splitting on a and applying the identity to *both* polarities:

$$\#SAT(F) = 2 \#SAT(F_{-a}) - (\#SAT(F_{-a} \wedge \neg F_a[a=0]) + \#SAT(F_{-a} \wedge \neg F_a[a=1])).$$

Each $\neg F_a[a=i]$ is a DNF. Splitting it by the *first violated clause* keeps the branches disjoint, so the counts simply add:

$$\neg(C_1 \wedge \dots \wedge C_t) = \bigvee_{i=1}^t (C_1 \wedge \dots \wedge C_{i-1} \wedge \neg C_i).$$

$$F = \underbrace{(\bar{a} \vee x_1 \vee x_2) \wedge (\bar{a} \vee x_3 \vee x_4) \wedge (a \vee x_1 \vee y)}_{F_a : \text{clauses containing } a} \wedge \underbrace{(x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_4 \vee y)}_{F_{-a}}$$



Either F has a short clause and we branch on it, or all clauses are long — and then a variable of low degree occurs only in long clauses. Either way the branching is strong.

Result

The number of satisfying assignments of a CNF formula over n variables of maximum variable degree Δ can be determined in time $\mathcal{O}^*(2^{(1-\frac{1}{3D})n})$, where $D = \max(\Delta, 3)$ — with *no* restriction on clause width.

Branching first on the high-degree variables extends this to any formula of constant clause density $\delta = m/n$, in time $\mathcal{O}^*(2^{(1-\frac{1}{c\delta})n})$.

3. Clause Learning $H = F_y$, y pure

Branching on the pure variable y gives the two formulas $F[y=1] = F_{-y}$ and $F[y=0] = F_{-y} \wedge G$, where $G = F_y[y=0]$. Instead of solving F_{-y} , we split it by G :

$$\#SAT(F_{-y}) = \#SAT(F_{-y} \wedge G) + \#SAT(F_{-y} \wedge \neg G).$$

In $\neg G$ we **learn clauses**, and the problem is rewritten as

$$\#SAT(F) = 2 \#SAT(F_{-y} \wedge G) + \#SAT(F_{-y} \wedge \neg G).$$

► Here $G = (a \vee x_1) \wedge (\bar{x}_1 \vee x_4)$, and the branch that carried no constraint gains three new 2-clauses:

$$\neg G \equiv (\bar{a} \vee x_1) \wedge (\bar{a} \vee \bar{x}_4) \wedge (\bar{x}_1 \vee \bar{x}_4).$$

A pure variable of low degree yields short learned clauses.

Result

Applied to the tight cases of the #3-SAT algorithm of [Kutzkov, 2007], clause learning gives a new record running time for deterministic exact model counting of 3-CNFs: #3-SAT is solvable in time $\mathcal{O}^*(1.637^n)$, improving $\mathcal{O}^*(1.6423^n)$.